

EnLang Master Rulebook

Universal Natural English Programming Language & Specification

EnLang Universal Master Specification & Rulebook (v1.0)

This is the complete, full-fledged master reference manual for EnLang a universal programming language featuring 100% Natural English syntax, multi-domain file extensions (.enlg, .enlgd, .enlgs, .enlgdb), integrated NLP capabilities, and clean Python transpilation.

Master Index

1. Language Philosophy
2. File Extensions Matrix
3. Variable Assignment & Binding
4. Data Types & Literals
5. Input & Output (I/O) Keywords
6. Arithmetic & Math Operators
7. Comparison & Relational Operators
8. Logical & Boolean Operators
9. Control Flow (Conditionals)
10. Loops & Iteration
11. Functions & Procedure Declarations
12. List & Collection Operations
13. Natural Language Processing (NLP) Primitives
14. Cybersecurity & Encryption Keywords
15. Database & SQL ORM Keywords (.enlgdb)
16. Script Automation Keywords (.enlgs)
17. Design & UI Layout Keywords (.enlgd)
18. Module & Package Imports
19. CLI Commands & Tooling

1. Language Philosophy

- Natural Readability First: Code reads like clean English sentences. Anyone who knows English can understand EnLang code immediately.
- Natural English Assignment Operator (store ... in ...): store is the explicit assignment operator, in connects to the target variable.
- Python Runtime Zero Overhead: All EnLang constructs compile directly into fast, clean Python code.
- Indent-Based Scoping: EnLang uses clean Python-style indentation for blocks.

2. File Extensions Matrix

Extension | Domain | Primary Purpose | Python Equivalent Execution

.enlg | Standard Logic | Algorithms, math, functions, business logic, NLP. | Standard Python (.py) script

.enlgd | UI & Design | Glassmorphic UI component markup. | Python HTML/CSS Layout String Generator

.enlgs | Script Automation | System automation, CLI tools, logging, timer pauses. | Python Automation (sys, datetime, time)

.enlgdb | Database & SQL | Table schema creation, record insertion, SQL queries. | Python SQLite3 ORM Engine

3. Variable Assignment & Binding

A. English Grammar Assignment Operator (store ... in ...)

EnLang Code:

```
store "racecar" in word
```

Python Equivalent:

```
word = "racecar"
```

B. Natural set Assignment

EnLang Code:

```
set score to 100
```

Python Equivalent:

```
score = 100
```

C. Natural is Assignment

EnLang Code:

```
word is "racecar"
```

Python Equivalent:

```
word = "racecar"
```

D. Standard = Assignment

EnLang Code:

```
word = "racecar"  
let word = "racecar"
```

Python Equivalent:

```
word = "racecar"
```

E. Increment / Decrement

EnLang Code:

```
increment score by 5  
decrement score by 1
```

Python Equivalent:

```
score += 5  
score -= 1
```

4. Data Types & Literals

Data Type | EnLang Literal Example | Python Equivalent Literal | Python Type

String | "Hello EnLang", 'Spandan' | "Hello EnLang", 'Spandan' | str

Integer | 42, -10, 0 | 42, -10, 0 | int

Float | 3.14159, 99.99 | 3.14159, 99.99 | float

Boolean | true, false | True, False | bool

List | [10, 20, 30], ["a", "b"] | [10, 20, 30], ["a", "b"] | list

Dictionary | {"key": "value"} | {"key": "value"} | dict

5. Input & Output (I/O) Keywords

A. Output (display, print, show, say, log)

EnLang Code:

```
display("Original Word: " + word)
print("Result: ", result)
show("Score: " + score)
say("Hello World")
log("System Initialized")
```

Python Equivalent:

```
print(str("Original Word: ") + str(word))
print("Result: ", result)
print(str("Score: ") + str(score))
print("Hello World")
print("System Initialized")
```

B. User Input (ask ... and store in ...)

EnLang Code:

```
ask "Enter your username: " and store in user_name
```

Python Equivalent:

```
user_name = input("Enter your username: ")
```

6. Arithmetic & Math Operators

A. Natural English Math Words

EnLang Code:

```
set total to price plus tax
set diff to price minus discount
set area to length times width
set average to sum divided by count
set rem to 10 modulo 3
set sq to 2 power of 8
```

Python Equivalent:

```
total = price + tax
diff = price - discount
area = length * width
average = sum / count
```

```
rem = 10 % 3
sq = 2  8
```

B. Standard Symbols (+, -, *, /, %,)

EnLang Code:

```
total = price + tax
area = length * width
```

Python Equivalent:

```
total = price + tax
area = length * width
```

7. Comparison & Relational Operators

Natural Phrase | EnLang Example | Python Equivalent

is equal to | if x is equal to y then: | if x == y:

is not equal to | if x is not equal to y then: | if x != y:

is greater than | if x is greater than y then: | if x > y:

is less than | if x is less than y then: | if x < y:

is greater than or equal to | if x is greater than or equal to y then: | if x >= y:

is less than or equal to | if x is less than or equal to y then: | if x <= y:

is in | if item is in list_var then: | if item in list_var:

is not in | if item is not in list_var then: | if item not in list_var:

is true | if is_valid is true then: | if is_valid == True:

is false | if is_valid is false then: | if is_valid == False:

8. Logical & Boolean Operators

A. Logical and, or, not

EnLang Code:

```
if age is greater than 18 and score is greater than 50 then:
display("Qualified")
```

Python Equivalent:

```
if age > 18 and score > 50:
print("Qualified")
```

9. Control Flow (Conditionals)

if / else if / else

EnLang Code:

```
if word is equal to reversed_word then:
display("Result: '" + word + "' IS a Palindrome!")
else if word is equal to "special" then:
display("Special case matched!")
else:
display("Result: '" + word + "' IS NOT a Palindrome.")
```

Python Equivalent:

```
if word == reversed_word:
    print(str("Result: ") + str(word) + str("' IS a Palindrome!"))
elif word == "special":
    print("Special case matched!")
else:
    print(str("Result: ") + str(word) + str("' IS NOT a Palindrome."))
```

10. Loops & Iteration

A. repeat N times:

EnLang Code:

```
repeat 5 times:
    display("Iteration")
```

Python Equivalent:

```
for _ in range(int(5)):
    print("Iteration")
```

B. for each <item> in <collection>:

EnLang Code:

```
set items to ["apple", "banana", "cherry"]
for each fruit in items do:
    display("Fruit: " + fruit)
```

Python Equivalent:

```
items = ["apple", "banana", "cherry"]
for fruit in items:
    print(str("Fruit: ") + str(fruit))
```

C. while <condition>:

EnLang Code:

```
set count to 0
while count is less than 5 do:
    display("Count: " + count)
    increment count by 1
```

Python Equivalent:

```
count = 0
while count < 5:
    print(str("Count: ") + str(count))
    count += 1
```

D. Loop Control (break, continue)

EnLang Code:

```
if count is equal to 3 then:
    break
```

Python Equivalent:

```
if count == 3:
    break
```

11. Functions & Procedure Declarations

A. Standard function Declaration

EnLang Code:

```
function calculate_discount(price, percentage):  
  set discount to price times percentage divided by 100  
  return price minus discount  
  
store calculate_discount(1000, 15) in final_price  
display("Final Price: ", final_price)
```

Python Equivalent:

```
def calculate_discount(price, percentage):  
    discount = price * percentage / 100  
    return price - discount  
  
final_price = calculate_discount(1000, 15)  
print("Final Price: ", final_price)
```

B. Concise func Declaration

EnLang Code:

```
func add_numbers(a, b):  
  return a plus b
```

Python Equivalent:

```
def add_numbers(a, b):  
    return a + b
```

12. List & Collection Operations

A. Creating Lists

EnLang Code:

```
set colors to ["red", "green", "blue"]  
create list numbers with 10, 20, 30
```

Python Equivalent:

```
colors = ["red", "green", "blue"]  
numbers = [10, 20, 30]
```

B. Adding / Appending to Lists

EnLang Code:

```
add "yellow" to list colors  
push 40 to list numbers
```

Python Equivalent:

```
colors.append("yellow")  
numbers.append(40)
```

13. Natural Language Processing (NLP) Primitives

A. Sentiment Analysis

EnLang Code:

```
analyze sentiment of "EnLang is fantastic!" and store in s_result
```

Python Equivalent:

```
from enlang_core.nlp_engine import analyze_sentiment
s_result = analyze_sentiment("EnLang is fantastic!")
```

B. Keyword Extraction

EnLang Code:

```
extract keywords from "AI is revolutionary" into keywords_list
```

Python Equivalent:

```
from enlang_core.nlp_engine import extract_keywords
keywords_list = extract_keywords("AI is revolutionary")
```

C. Text Similarity

EnLang Code:

```
calculate similarity between "hello" and "hi" and store in sim_score
```

Python Equivalent:

```
from enlang_core.nlp_engine import calculate_similarity
sim_score = calculate_similarity("hello", "hi")
```

14. Cybersecurity & Encryption Keywords

A. SHA-256 Hashing

EnLang Code:

```
hash "MySecret123" with sha256 and store in hash_val
```

Python Equivalent:

```
import hashlib
hash_val = hashlib.sha256(str("MySecret123").encode('utf-8')).hexdigest()
```

B. MD5 Hashing

EnLang Code:

```
hash "MySecret123" with md5 and store in hash_md5
```

Python Equivalent:

```
import hashlib
hash_md5 = hashlib.md5(str("MySecret123").encode('utf-8')).hexdigest()
```

15. Database & SQL ORM Keywords (.enlgdb)

EnLang Code (.enlgdb):

```
connect to database "app.db" as main_db
```

```
define table users with columns id as integer, username as text, email as text
insert record into users with values 101, 'Spandan', 'spandan@enlang.org'
execute query "SELECT * FROM users" on database main_db and store in all_users
```

Python Equivalent:

```
import sqlite3
main_db = sqlite3.connect("app.db")

_cur = main_db.cursor()
_cur.execute('CREATE TABLE IF NOT EXISTS users (id integer, username text, email text)')
main_db.commit()

_cur = main_db.cursor()
_cur.execute(f'INSERT INTO users VALUES (101, \'Spandan\', \'spandan@enlang.org\')')
main_db.commit()

_cur = main_db.cursor()
_cur.execute("SELECT * FROM users")
all_users = _cur.fetchall()
main_db.commit()
```

16. Script Automation Keywords (.enlgs)

EnLang Code (.enlgs):

```
log message "Automated Security Task Initialized"
parse argument target_ip into host_name
pause script for 2 seconds
```

Python Equivalent:

```
import datetime
print(f'[{datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S")}] SCRIPT LOG: Automated Security Task Initialized')

import sys
host_name = sys.argv[1] if len(sys.argv) > 1 else 'target_ip_default'

import time
time.sleep(float(2))
```

17. Design & UI Layout Keywords (.enlgd)

EnLang Code (.enlgd):

```
create card named profile_card with title "User Profile Dashboard"
create button named save_btn with label "Save Changes" and action "alert('Saved!')"
render layout with profile_card, save_btn
```

Python Equivalent:

```
profile_card = f"""<div class="card-enlgd"><h2>User Profile Dashboard</h2></div>"""
save_btn = f"""<button class="btn-enlgd" onclick="alert('Saved!')">Save Changes</button>"""
print('<div class="layout-enlgd">' + ''.join([profile_card, save_btn]) + '</div>')
```

18. Module & Package Imports

EnLang Code:

```
import module random
import math as m
from datetime import datetime
```

Python Equivalent:

```
import random
import math as m
from datetime import datetime
```

19. CLI Commands & Tooling

enlang run file.enlg	# Runs EnLang source script directly
enlang build file.enlg	# Compiles EnLang code to standalone Python file (.py)
enlang repl	# Starts Interactive NLP Natural English Shell
enlang version	# Prints version info and supported extensions